Laborator nr. 2

Comenzi Unix/Linux uzuale. Expresii regulate. Editoare de texte. Shell. Fişiere de comenzi.

1. Comenzi Unix/Linux uzuale

Vom prezenta, în ordine alfabetică, cele mai uzuale comenzi Unix/Linux

apropos Cauta cuvantul specificat în baza de date *whatis* ce contine scurte descrieri despre comenzile Unix și funcțiile de biblioteca C, și afișarea rezultatului căutării (sunt afișate toate potrivirile, nu doar cele exacte, ale cuvantului căutat).
Sintaxa:
`apropos cuvint`

Exemplu:

```
Shell
$apropos write
```

cat Concateneaza și afiseaza continutul fisierelor.
Sintaxa: `cat [optiuni] fisiere`
Opțiuni:
-n afișează și numărul liniei
-v Tipareste caracterele de control (^caracter).

Exemplu:

```
Shell
cat f1 f2 f3
```

concateneaza fișierele f1,f2,f3 și afișează la terminal rezultatul.

cc Compilatorul programelor C (C Compiler).
Sintaxa: `cc [optiuni] fisier(e).[c|s|o]`
Comanda lansează în execuție compilatorul limbajului C primind ca intrare textul sursa în unul sau mai multe fisiere sursa C (cu extensia .c), in asamblare (.s) sau obiect (.o). Compilatorul C este cu doua treceri. Prima trecere produce un cod intermediar C pe baza textului sursa. Acest cod este translatat în limbaj de asamblare de trecerea a doua a compilatorului. Codul intermediar este independent de masina pe care este instalat sistemul, în timp ce limbajul de asamblare, asamblorul și loader-ul depind de hardware.
Opțiuni:
-c Compilare simpla (fără a genera cod executabil), produce fișierele obiect corespunzătoare fișierelor ce urmează opțiunii.
-o f Generează fișierul executabil *f*. Dacă aceasta optiune lipsește, programul executabil va purta numele *a.out*.
-f Compilarea cu opțiunea de virgula mobilă.
-lm Legarea bibliotecii matematice.

cd Schimba directorul curent.

chmod Schimba drepturile de acces la un fisier ordinar sau director (Prezentată în lucrarea anterioara).

cmp Compara doua fișiere.
Sintaxa: `cmp [optiuni] fisier1 fisier2`
Opțiuni:
-l Afișează numărul și valoarea în octal pentru fiecare diferenta intalnita.
-s Rezultatul comparării este dat printr-un cod (0=identice, 1=diferite, 2=eroare).

cp Copiază un fișier în altul sau fisiere intr-un director.
Sintaxa: `cp fisier1 fisier2`

```shell
$ cp fisiere director
```

diff Comanda afișează diferențele a doua fișiere sau directoare
Sintaxa: `diff [optiuni] FILES`

```shell
$ diff fisier1 fisier2
```

echo Afișează argumentele sale la terminal. Comanda se utilizează în special în fișiere de comenzi pentru a afișa diferite mesaje.

ls Listeaza continutul directoarelor (Prezentată în lucrarea anterioara).

mail Trimite un mesaj unui utilizator sau vizualizeaza conținutul “cutiei poștale” a utilizatorului care emite comanda.
Sintaxa: `mail utilizator`
Utilizatorul se specifica prin numele sau de login.
Utilizarea comenzii pentru inspectarea cutiei poștale determina afișarea unui mesaj și a unui prompter. La acest prompter, utilizatorul răspunde cu subcomenzile:
<CR> sau
+ Următorul mesaj.
- Afișează precedentul mesaj.
d Șterge mesajul curent și afișează următorul.
p Reafiseaza mesajul curent.
s f Salveaza mesajul în fișierul specificat.
q Quit *mail* cu păstrarea mesajelor nesterse.
x Exit *mail* și pastreaza nemodificata cutia poștală.
!cmd Executa comanda Unix *cmd*.
Exemplu:

```shell
$ mail user1
Mai faceti modificari in proiect? Astept raspuns !
.
$
```

man Afiseaza continutul manualului programatorului Unix, legat de un anumit subiect.
Sintaxa: `man subiect`

Exemplu:

```
Shell
man ls
```

Afișează informații despre comanda ls.

mesg Stabilește modul de tratare al mesajelor.
Sintaxa: `mesg [optiuni]`
Opțiuni:
-y Permite recepția și afișarea mesajelor.
-n Invalidează recepționarea mesajelor.

make Utilitarul make a fost conceput în scopul automatizării părții de rutina a activității de întreținere a programelor. Ideea de baza este memorarea într-un fișier cu o structura particulara, numit implicit *makefile*, a interdependentei programelor si a secvențelor de comenzi necesare pentru obtinerea programului executabil din textele sursa ale componentelor de baza.
Sintaxa:
`make [optiuni][-f makefile][macrodef...][fisiere_tinta]`
Opțiuni:
-n Se afișează liniile de comanda, dar nu se executa.
-t Actualizarea fișierelor țintă.
-p Afișare macrodefinitii.
-f Precede numele unui fișier de descriere (tip makefile).
Exemplu: Programul *prg* este generat prin compilarea și legarea modulelor sursa *x.c*, *y.c* și *z.c* folosind biblioteca *ls*. Modulele obiect se vor numi respectiv *x.o*, *y.o* și *z.o*. Se considera ca modulele *x.c* și *y.c* includ fișierul *def.h*. Interdependența dintre module este descrisă prin:

```
Shell
prg: x.o y.o z.o
cc x.o y.o z.o -ls -o prg
x.o y.o: def.h
```

Dacă acest fișier este denumit *Makefile*, la fiecare lansare a comenzii **make** se va obține o varianta reactualizata a programului *prg* pe baza ultimelor versiuni din surse. Dacă nu exista variante mai noi ale fișierelor componente în raport cu ultima compilare a lui *prg* nu se va face nici o compilare. Dacă însă între timp s-ar fi modificat *def.h*, s-ar fi recompilat *x.c* si *y.c* (nu si *z.c*). Dacă în linia de comanda make nu se specifica nici un fișier, atunci primul fișier menționat în lista de descrieri este automat recompilat. Dacă însă, folosind fișierul makefile de mai sus, am scrie: *make x.o* ar avea loc recompilarea lui *x.c* conditionata de modificarea între timp a lui *x.c* sau *def.h*. O facilitate utila este folosirea macrodefinitiilor. În acest caz numele unei macrodefinitii începe cu semnul **$** urmat de unul sau mai multe caractere între paranteze. Pentru exemplul de mai sus:

```Shell
OB = x.o y.o z.o
BIBL = -ls
prg: $(OB)
cc $(OB) $(BIBL) -o prg
```

În interiorul fișierelor de descriere (makefile) se pot introduce comentarii precedate de caracterul '#'.

mkdir Creeaza un director. Intrările `.` si `..` sunt create automat. Drepturile de acces sunt cele implicite (0777). Se impune dreptul de a scrie]n director pentru a crea directoare.
Sintaxa: `mkdir [-m acces][-p] directoare`
Opțiuni:
-m acc Specifica drepturile de acces pentru directorul creat;
-p Creeaza toate directoarele inexistente din numele de cale specificat, dacă acestea nu exista.
Exemplu:

```Shell
$ mkdir -p /home/student/prj
```

Creeaza subdirectoarele indicate;

```Shell
mkdir grp1 grp2 grp3
```

Creează 3 directoare noi;

mv Redenumeste un fisier sau muta un fisier intr-un director.
Exemplu:

```Shell
mv f1 f2
```

Fișierul f1 este redenumit în f2.

```Shell
mv f ./d1
```

Fișierul f este copiat în directorul d1.

pwd Afișează numele complet al directorului curent.
rm Șterge un fișier sau director.
Sintaxa: `rm [optiuni] fisier(e)`
Opțiuni:
-i Cere permisiunea înainte de a șterge fișierul.
-f Forțează ștergerea, neraportand eventualele erori.
-r Șterge toate fisierele din director și directorul.

Exemplu:

```
Shell
rm -r project
```

rmdir Șterge un director dacă acesta este vid.

test Evaluează expresia ce urmează comenzii. Comanda este utilizata frecvent în fișiere de comenzi. Rezultatul evaluării este true sau false. Comenziile de mai jos sunt adevărate:

`test -r f`	Dacă f există și are permisiunea read.
`test -w f`	Dacă f există și are permisiunea write.
`test -x f`	Dacă f există și are permisiunea execute.
`test -d f`	Dacă f există și este director.
`test -f f`	Dacă f există și este fisier ordinar.
`test -s f`	Dacă f există și are dimensiunea > 0.
`test -z s1`	Dacă lungimea șirului s1 este 0.
`test -n s1`	Dacă lungimea șirului s1 nu este 0.
`test s1=s2`	Dacă s1 și s2 sunt identice.
`test s1!=s2`	Dacă s1 și s2 nu sunt identice.
`test s1`	Dacă șirul s1 nu este șirul null.
`test n1 -eq n2`	Dacă întregii n1 si n2 sunt egali.

Alți operanzi pentru comparare: `-ne`, `-gt`, `-ge`, `-lt`, `-le`.
Se pot utiliza operatori logici `!`(negație), `-a` (și), `-o` (sau) și `()` (paranteze rotunde) pentru a construi expresii compuse din cele de mai sus.

tty Afișează numele complet al terminalului de la care s-a emis comanda.

wc Contorizarea numărului de linii, cuvinte și caractere dintr-un fișier.
Sintaxa: `wc [optiuni] [fisier]`
Opțiuni:
- -l Afișează numărul liniilor din fișier.
- -w Afișează numărul cuvintelor din fișier.
- -c Afișează numărul caracterelor din fișier.

Exemplu:

```
Shell
wc file.txt
```

who Afișează numele utilizatorilor conectați în sistem.

write Trimite mesaj unui alt utilizator conectat la sistem;
Sintaxa: `write utilizator [terminal]`
Mesajul care se trimite se specifica la fel ca in cazul lui *mail*.

Pentru alte comenzi Linux/Unix a se vedea **bibliografia**.

2. Expresii regulate

Expresiile regulate au rolul de a specifica mulțimi de șiruri de caractere. Ele stabilesc un șablon pentru potriviri de text. Sunt utilizate în general pentru a identifica dintr-un text anumite șiruri de caractere. Expresiile regulate descrise aici sunt recunoscute de majoritatea programelor utilitare. Editorul **vi** permite facilități suplimentare.

În general orice caracter ce apare într-o expresie regulată identifica un caracter din text. Există însă anumite caractere speciale care permit definirea de sabloane variabile. In expresiile regulate se utilizează două tipuri de caractere:
- caractere alfanumerice, care desemnează exact caracterele reprezentate;
- caractere speciale sau operator care desemnează repetitii, alegeri, opțiuni, etc.

Caractere speciale:

`\` Caracter de protejare - anuleaza semnificația speciala a caracterului care il urmează.

`.` Caracter arbitrar (orice caracter).

`*` `+` Operatori de repetitie:

`a*` orice număr de caractere a, chiar 0;

`a+` orice număr de caractere a, cel puțin unul.

`$` La sfarsitul unei expresii regulate marchează un sfarsit de linie.

`^` La începutul unei expresii regulate marchează început de linie.

`[s]` Desemnează o mulțime de caractere. În mulțimea *s* singurele caractere speciale sunt: -,] și ^. De exemplu:

`[abc]` Unul dintre caracterele a, b sau c.

`[A-Za-z]` O litera.

Caracterul '^' complimenteaza o mulțime dacă este la început. De exemplu:

`[^0-9]` Caracter nenumeric.

O secvență de caractere dintr-o expresie regulată se poate delimita prin '`\(`' si '`\)`'. Construcția este numita expresie regulată marcată. Aceasta expresie poate fi referita apoi prin '`\i`' cu `i` între 1 și 9. De exemplu, expresia:

`\([a-z]*\)\&\1` Selectează șirul 'adi\&adi'.

Pentru expresii regulate marcate utilitarele **awk** si **egrep** accepta construcții de forma:

`\(expr\)*` Zero sau mai multe apariții ale expresiei regulate

`\(expr\)+` Una sau mai multe apariții ale expresiei regulate.

`\(expr\)?` Zero sau o apariție a expresiei regulate.

Doua expresii regulate separate prin '`|`' semnifica una dintre ele (alternativa). De exemplu:
`(ab|cd+)?(ef)*`
Pe prima poziție apare sau nu *ab* sau *c* urmat de cel puțin un *d*, și urmat de oricate grupuri de *ef*.

Expresiile regulate între paranteze simple realizeaza o grupare. De exemplu:

`(a|b)(c|d)` Identifica *ac*, *ad*, *bc* sau *bd*.

`"` Toate caracterele situate între `"` își pierd semnificația speciala.

`/` Indica contextul următor. De exemplu:

`ab/cd` *ab* doar dacă este urmat de *cd*.

{} Semnifica repetiții ale unei expresii. De exemplu:

`a\{1,5\}` Una pana la cinci apariții ale lui *a*.

`exp\{6,\}` Cel puțin 6 apariții ale expresiei *exp*.

`exp\{2\}` Exact doua apariții ale expresiei *exp*.

`[0-9]\{2,\}` Secventa din doua sau mai multe cifre.

Dacă mai multe caractere speciale apar într-o expresie regulată se aplica regula de precedenta. Conform acesteia, precedenta cea mai mare o are `[]` după care grupul `*`, `+` si `?` și ultimul este `|`.

La inlocuirea șirurilor de caractere specificate prin expresii regulate caracterul '&' identifica expresia regulată. Pentru a genera caracterul '&' acesta trebuie prefixat de \. Caracterul '\' urmat de <Enter> generează o linie nouă.

3. Editorul de texte VI

Editorul **vi** permite vizualizarea conținutului unui fișier prin deplasarea unei ferestre de-a lungul textului fișierului. Editorul are trei moduri de lucru: **modul comanda**, **modul inserare** si **modul ex**. Diferențele dintre aceste moduri rezulta din analiza comenzilor ce urmează a fi prezentate.

Apelul editorului se face sub forma:

vi [-r | -R] [+pozitionare] [fisier]

unde:

-r Încarcă ultima versiune salvată înainte de căderea sistemului.

-R Se interzice modificarea conținutului fișierului.

Daca se specifica *pozitionare* cursorul se pozitioneaza pe linia specificata (+*nr_linie*) sau pe șirul căutat (+/*sir_cautat*).

Revenirea în sistem se realizeaza prin comanda **ZZ** care face salvare pe disc doar dacă au avut loc modificări, sau prin **:wq** (Write, Quit) care realizeaza salvarea bufferului de lucru pe disc. Dacă nu se dorește salvarea se folosește **:q!** (Quit).

3.1. Comenzi pentru editorul vi:

a) Comenzi de deplasare ecran

Ctrl/D Deplasare jos 1/2 ecran. **Ctrl/F** Deplasare jos 1 ecran.

Ctrl/U Deplasare sus 1/2 ecran. **Ctrl/B** Deplasare sus 1 ecran.

b) Comenzi de deplasare cursor în ecran

Daca tastatura nu contine tastele de direcție deplasarea cursorului se poate face prin:

-, **H** Cursor pe linia dinainte.

k Cursor pe linia dinainte pastrand coloana.

+, **L** Cursor pe linia următoare.

j Cursor pe linia următoare pastrand coloana.

M Cursor la mijlocul ecranului.

nG Cursor pe linia n.

<Bksp>, **h** Cursor cu un caracter spre stanga.

<sp>, **l** Cursor cu un caracter spre dreapta.

$ Cursor la sfarsit de linie.

^ Cursor la început de linie (caracter neblanc).

0 Cursor pe primul caracter din linie.

b, **B** Cursor pe cuvantul precedent.

w, **W** Cursor pe cuvântul următor.

e, **E** Cursor la sfarsitul cuvantului.

c) Comenzi de stergere si inlocuire

x/**X** Șterge caracterul de sub sau dinaintea cursorului.

dw Șterge cuvantul curent.

dd Șterge linia.

p/**P** Reface o ștergere anterioara.

cc...Esc Înlocuiește o linie completa.

R...Esc Înlocuiește mai multe caractere.

rx Înlocuiește caracterul de sub cursor cu *x*.

d) Comenzi diverse

mx Marchează linia cu *x*.

/ Căutare înainte.

? Căutare înapoi.
u Refacerea stării dinaintea ultimei modificări.
yw Trimite cuvantul curent într-un buffer.
yy Trimite linia curentă într-un buffer intern.
Lyy Trimite linia curentă în buffer *L*. De exemplu:
cyy Trimite linia curenta in buffer *c*.
c10y Trimite 10 linii în buffer *c*.
LP Depune conținutul bufferului *L* înaintea poziției cursorului. Dacă bufferul conține o linie întreaga ea este inserată înaintea liniei curente.
Lp Depune conținutul bufferului după poziția cursorului. Dacă bufferul contine o linie întreaga ea este inspirată după linia curentă.

3.2. Comenzile modului inserare

În modul inserare se intra folosind caracterele **i,I,a,A,o,O** si dupa introducerea textului dorit se revine în editor prin tasta **Esc**.

i/**I**...Esc Inserare înaintea cursorului sau la începutul liniei curente.
a/**A**...Esc Inserare după cursor sau la sfarsitul liniei curente.
o/**O**...Esc Inserare după sau înaintea liniei curente.

3.3. Comenzile modului ex

Se prefixeaza comanda cu caracterul **:**. La execuția unei astfel de comenzi ea este afișată pe ultima linie a ecranului curent.

:n**r** fis (Read) Plasarea fișierului *fis* în bufferul de editare pe linia *n*.
:**w** fis (Write) Scrierea conținutului bufferului de editare în fișierul *fis*.
:**!** cmd Parasirea temporara a editorului **vi** si executarea comenzii specificate.

La editarea programelor C se poate configura editorul **vi** folosind comanda **set** (pentru indentare automată (ai) și eventual pentru fixarea lungimii tab-ului (ts)). Se pot defini și macrouri.

Pentru detalii se recomanda **man vi** sau **man ex**.

4. Shell. Fişiere de comenzi.

Shell-ul este un program care permite utilizatorilor să tasteze şi execute comenzi. Shell-ul reprezinta in acelasi timp **limbajul de comanda** al sistemului de operare Unix și **un limbaj de programare**:

- limbaj de comanda: shell-ul reprezinta interfata dintre utilizator și sistemul de operare. Se permite definirea unor comenzi complexe cu ajutorul comenzilor elementare, ceea ce permite extinderea mediului de operare existent.
- limbaj de programare: sunt introduse noțiunile de variabilă, parametru, funcție si structuri de control specifice.

4.1. Comenzi shell

O comanda transmisa shell-ului pentru interpretare are formatul:

```shell
command arguments
```

unde `command` este numele programului care se executa și `arguments` argumentele sale. Mai multe comenzi scrise pe o linie trebuie separate prin `;` . Orice

comanda executată întoarce un număr ce indica starea de ieșire din comanda. **În Unix convenția pentru starea de ieșire este 0 pentru o comanda executată cu succes și diferit de 0 în caz de eșec**. Comenzile pot fi conectate prin **pipe** (simbolul `|`) astfel incat **ieșirea unei comenzi constituie intrare pentru a doua**. În acest caz starea de ieșire este a ultimei comenzi din pipe.

Exemplu:

```
Shell
ls -l | more
```

aplica filtrul `more` pe rezultatul comenzii `ls`.

Dacă secvența de comenzi este terminata cu ampersand (`&`), ele se executa asincron in background. Shell-ul afișează identificatorul procesului lansat.

Continuarea unei comenzi pe linia următoare este posibilă dacă linia este terminata cu backslash (`\`).

Caracterele `&&` executa comanda de după ele dacă precedenta comanda a fost executată cu succes. În caz contrar se pot folosi caracterele `||`. De exemplu:

```
Shell
who | grep "student" > /dev/null && echo "student's logged on"
```

O linie shell ce începe cu `#` este considerată comentariu.

4.2. Variabile de mediu/variabile shell

Variabilele de mediu sunt un set de valori dinamice cu nume, valori ce pot afecta modul in care se comporta procesele care rulează pe un computer.

Ca orice limbaj de programare, limbajul shell-ului conţine variabile. Variabilele shell pot fi: **variabile utilizator**, **parametri pozitionali, variabile predefinite si variabile speciale**.

4.2.1. Variabile utilizator

Variabilele utilizator sunt definite prin:

`var1=val1 var2=val2 ...`

Continutul variabilei poate fi accesat prin prefixarea numelei variabilei cu caracterul $.

Exemplu:

```
Shell
loc=/home/student
cd $loc
```

Variabilele utilizator sunt evaluate la valoarea lor, în afara cazului în care valoarea este delimitată de caracterul ' (apostrof).

Numele unei variabile nu este permis sa fie egal cu numele unei funcții. Dacă se dorește definirea unei variabile noi prin concatenare unui nume de variabile cu un șir de caractere fix, numele variabilei trebuie delimitat de caracterele `{` și `}`. De exemplu:

```Shell
$ num=3
$ k=${num}tmp
$ echo $k
3tmp
```

Shell-ul oferă un mecanism de substituție bazat pe următoarele caractere:

'...' Nici un caracter delimitat între apostrofuri nu are semnificație specială.

"..." Nici un caracter delimitat de ghilimele nu are semnificație specială cu excepția: $, ` si \.

\c Nu interpretează caracterul **c**. Între ghilimele este caracter de evitare pentru: $, `, " și \. În alte cazuri este ignorat.

Exemplu:

`cmd` - executa comanda **cmd**.

```Shell
$ rep=`pwd`
```

- atribuie variabilei `rep` rezultatul comenzii `pwd`.

La intrarea în sistem fiecare utilizator are o copie a programului shell. Acest shell pastreaza un mediu distinct pentru fiecare utilizator din sistem. De exemplu:

```Shell
$ cat test
x=50
echo “$x”
$ x=100
$ test
50
$ echo $x
100
```

Acest exemplu demonstrează că mediul shell-ului inițial contine variabila **x** ce are valoarea 100 și care nu este modificata la 50 de către programul shell **test**. Explicația este simplă, deoarece shell-ul inițial lansează un subshell pentru execuția programului **test**, fiecare shell avand propria variabila **x**.

Comanda export

Pentru a face cunoscută o variabila unui subshell se folosește comanda **export**.

1. Orice variabila care nu este importantă este locală si nu este cunoscuta subshell-ului.
2. Variabilele exportate și valorile lor sunt copiate în mediul subshell-ului, unde pot fi accesate și modificate. Efectul modificări nu este vizibil in shell-ul părinte.
3. Dacă un subshell exportă explicit o variabila a cărei valoare a modificat-o ea se transmite. Dacă subshell-ul nu exportă explicit o variabila a cărei valoare a modificat-o modificarea afectează doar variabila locală, indiferent daca variabila a fost exportata de shell-ul parinte.
4. O variabila poate fi exportată înainte sau după atribuirea unei valori.

Exemplu:

```shell
$ y=10
$ cat test1
echo x = $x
echo y = $y
$ export y
$ test1
x =
y = 10
$
```

4.2.2. Parametri pozitionali

Parametrii pozitionali `$1,... $9` sunt argumentele din linia de comanda, iar `$0` este numele programului shell ce se executa. Parametrii pozitionali pot fi actualizați prin comanda **`set`**.

4.2.3. Variabile predefinite si variabile speciale

Exista un număr de variabile predefinite folosite de interpretor sau de programele de deschidere a sesiunii. Acestea sunt:

`$HOME` - Desemnează directorul implicit pentru comanda **`cd`**.

`$PATH` - Defineste lista directoarelor parcurse de shell în căutarea unui fișier executabil corespunzător comenzii introduse (directoarele sunt separate prin :).

`$IFS` - Multimea caracterelor separatoare (blanc (040), \t (011), \n (012)), folosite de shell la analiza liniei de comanda.

`$PS1` - Defineste prompterul shell, implicit **$**.

`$PS2` - Defineste prompterul liniilor de continuare, implicit **>**.

`$SHELL` - Indica numele shell-ului curent.

Shell-ul atribuie valori implicite variabilelor: `IFS`, `PATH`, `PS1`, `PS2` la intrarea in sesiune. Programul **login** inițializează cu valori implicite variabila `HOME`.

Variabilele speciale sunt **read-only**:

$# - numărul argumentelor din linia de comanda (exclusiv `$0`).

$* - argumentele `$1, $2,..., $9` concatenate.

$@ - argumentele `$1, $2,...$9` separate.

$? - starea de ieșire a ultimei comenzi efectuate.

$$ - identificatorul de proces asociat shell-ului.

$- - flagurile actuale ale shell-ului pozitionate prin **`set`**.

$! - identificatorul ultimului proces lansat in background.

4.3. Programe (proceduri) shell

Posibilitatea de a construi proceduri alcătuite din comenzi ale sistemului de operare constituie una din principalele facilități puse la dispoziție de către shell. Interpretorul permite execuția unor fisiere de comenzi tratate ca proceduri shell. Apelul unei proceduri este identic cu al unei comenzi:

```shell
$procedura arg1 arg2 ... argn
```

Procedura corespunde unui fișier de comenzi. Transmiterea parametrilor unei proceduri se face prin valoare. Apelul unei proceduri shell genereaza un proces shell fiu pentru citirea și execuția comenzilor. Execuția unui fișier de comenzi se face prin:

```Shell
$ sh fis_com [ parametrii]
```

sau

```Shell
$ fis_com [ parametrii]
```

Forma a două dacă fișierul desemnat, **`fis_com`**, are pozitionat bitul de execuție din drepturile de acces (comanda **`chmod u+x fis_com`**).

Parametrilor actuali specificati în linia de apel le corespund parametri formali `$1`, `$2, ... $9` din interiorul procedurii. Numele fisierului de comenzi este referit în interior acesteia prin `$0`.

Procedurile shell pot fi apelate recursiv.

4.4. Funcții și comenzi incorporate în shell

Incepand cu versiunea 2 din Unix System V în programe shell se pot defini funcții. Formatul general pentru definirea unei funcții este:

```Shell
func_name()
{ cmd1; ... cmd2; }
```

unde *`func_name`* este numele funcției, parantezele marchează definirea funcției, iar între acolade este specificat corpul funcției. Se impune ca prima comanda sa fie separată de acolada cu un blanc, iar ultima comanda sa fie terminata cu caracterul ';' dacă acolada se afla pe aceeași linie cu comanda. De regula, dacă un utilizator și-a definit mai multe funcții într-un fișier, *`myfuncs`*, el poate face cunoscut shell-ului curent aceste funcții prin:

```Shell
. myfuncs
```

De exemplu, o funcție utilă este cea de schimbare a directorului curent. Aceasta se bucura de proprietatea ca ea se executa în mediul curent. Definitia ei, continuta in fisierul `myfuncs`, este:

```Shell
mycd()
{
    crtdir=`pwd`
    if [ "$1" = "" ]
    then
       echo $olddir
       cd $olddir
```

```
    else
       cd "$1"
    fi
    olddir=$crtdir
}
```

Pentru a ilustra folosirea acestei funcții se poate folosi secvența de comenzi:

```Shell
$ . myfuncs
$ pwd
/root/student
$ mycd /home/student
$ pwd
/home/student
$ mycd -
/root/student
```

Comparativ execuția unei funcții este mai rapidă decat a unui program shell echivalent, deoarece shell-ul nu necesita căutarea programului pe disc, deschiderea fișierul și încărcarea conținutul sau în memorie.
Ștergerea unei definiții de funcții este similară cu ștergerea unei variabile. Se folosește comanda **`unset`** *`func_name`*.

Comenzile incorporate în shell pot fi apelate direct in programele shell. Lista și efectul lor este prezentat în cele ce urmează (lista completa se poate afla rulând comanda **man bash**):

`:` Comanda fără efect.

`. file` Se citesc și se executa comenzile din fișierul *file* în mediul curent. Shell folosește variabila `PATH` pentru căutarea fișierului *file*. Fișierul nu trebuie sa fie executabil.

`break` `[n]` Comanda de părăsire a celei mai interioare bucle *for*, *while* sau *until* ce contine *break*. Dacă *n* este specificat se iese din *n* bucle. De exemplu:

```Shell
while true
do
       cmd=`getcmd`
       if [ "$cmd" = quit ]
       then break
       else processcmd "$cmd"
       fi
done
```

`cd` `[dir]` Schimba directorul curent la cel specificat. Directorul curent este parte a mediului curent. Din acest motiv la execuția unei comenzi **cd** dintr-un subshell doar directorul curent al subshell-ului este modificat. Exista variabila `$PATH` (Unix System V), care permite căutări suplimentare pentru comanda **cd**. De exemplu:

```
Shell
$ echo $PATH
:/home/student:/home/student/lab
$ cd l1
/ home/student/lab/l1 # avand in vedere ca l1 este subdirector pentru ultima cale.
```

continue [n] Comanda permite trecerea la o nouă iterație a buclei *for*, *while* sau *until*. De exemplu:

```
Shell
for file
do
if [ ! -f "$file" ]
then
       echo "$file not found"
       continue
fi
# prelucrarea fisierului
done
```

echo [-n][arg] Comanda de afișare a argumentelor la ieșirea standard. Dacă opțiunea *-n* este specificata caracterul '\n' nu este scris la ieșirea standard. De exemplu:

```
Shell
$ echo 'X\tY'
X Y
```

eval cmd Evalueaza o comanda si o executa. De exemplu:

```
Shell
$ x='ab cd'
$ y='$x'
$ echo $y
$x
$ eval echo $y
ab cd
```

Se observa ca **eval** parcurge lista de argumente de doua ori: la transmiterea argumentelor spre **eval** și la execuția comenzii. Lucrul acesta este ilustrat de exemplul:

```
Shell
$ pipe="|"
$ ls $pipe wc -l
| not found
wc not found
-l not found
```

Rezultatul dorit se obține prin:

```
Shell
$ eval ls $pipe wc -l
5
```

Comanda **eval** este folosită în programe shell care construiesc linii de comanda din mai multe variabile. Comanda e utila daca variabilele contin caractere care trebuie sa fie recunoscute de shall nu ca rezultat al unei substituții. Astfel de caractere sunt: ;, |, &, < , >, ".

exec `prg` Execută programul, *prg*, specificat. Programul lansat in executie înlocuiește programul curent. Dacă **exec** are ca argument redirectarea I/O, atunci shell-ul va avea I/O redirectate. De exemplu:

```
Shell
# contorizeaza numarul de linii dintr-un fisier
file=$1
count=0
exec < $file
while read line
do
       count=`expr $count + 1`
done
echo $count
```

exit `[(n)]` Cauzează terminarea shell-ului curent cu cod de ieșire egal cu *n*. Dacă *n* este omis, codul de ieșire este cel al ultimei comenzi executate.

export `[v...]` Marchează *v* ca nume de variabila exportată pentru mediul comenzilor executate secvențial (pentru subshell). Dacă nu este precizat nici un argument se afișează o listă cu toate numele exportate în acest shell. Numele de funcții nu poate fi exportat.

getopts `opt v` Comanda este folosită la prelucrarea opțiunilor din linia de comanda. Se executa, de regula, in bucle. La fiecare iterație **getopts** inspectează următorul argument din linia de comanda și decide dacă este o opțiune validă sau nu. Decizia impune ca orice opțiune sa inceapa cu caracterul '-' și să fie urmată de o litera precizata in *opt*. Dacă opțiunea există si este corect precizata, ea este memorată în variabila *v* și comanda returnează zero. Dacă litera nu este printre opțiunile precizate în *opt*, comanda memorează în *v* un semn de întrebare și returnează zero cu afișarea unui mesaj de eroare la standardul de eroare.
În cazul în care argumentele din linia de comanda au fost epuizate sau următorul argument nu începe cu caracterul '-' comanda returnează o valoare diferită de zero.
De exemplu, pentru ca **getopts** sa recunoască opțiunile -a și -b pentru o comanda oarecare, *cmd*, apelul este:

```shell
getopts ab var
```

Comanda *cmd* se poate apela:
`cmd -a -b` sau `cmd -ab`
În cazul în care opțiunea impune un argument suplimentar acesta trebuie separat de opțiune printr-un blanc. Pentru a indica comenzii **`getopts`** ca urmează un argument după o opțiune, litera opțiunii trebuie postfixata cu caracterul ':'. De exemplu, dacă opțiunea b, din exemplul anterior, ar necesita un argument:
`getopts ab: var`
Dacă **`getopts`** nu găsește după opțiunea b argumentul în variabila **`var`** se memorează un semn de întrebare și se va afișa un mesaj de eroare la ieșirea standard.
În caz ca argumentul exista el este memorat intr-o variabila speciala `OPTARG`. O alta variabila speciala, `OPTIND`, este folosită de comanda pentru a preciza numărul de argumente prelucrate. Valoarea ei inițială este 1.

`hash` `[-r][cmd]` Permite căutarea unor comenzi și memorarea locului unde se găsesc acestea în structura arborescentă de fișiere. Dacă argumentul lipsește sunt afișate toate comenzile memorate. Dacă opțiunea -n este specificata se șterg comenzile memorate.

`pwd` Cauzează afișarea directorului curent.

`read [v...]` Se citește o linie din fișierul standard de intrare și se atribuie cuvintele citite variabilelor **v** ... (dacă `IFS` nu a fost redefinită). De exemplu:

```shell
$ IFS=:
$ read x y z
123:345:678
$ echo $x
123
```

`readonly [v...]` Identică cu **`read`**, dar valoarea variabilei **v** nu poate fi schimbată prin atribuiri ulterioare. Dacă argumentul lipsește se afișează variabilele read-only.

`return` `[n]` Permite revenire dintr-o functie cu valoarea **n**. Dacă **n** este omis, codul returnat este cel al ultimei comenzi executate. Valoarea returnată poate fi accesată prin variabila `$?` și poate fi testată în structurile de control **`if`**, **`while`** și **`until`**. De exemplu:

```shell
testare()
{
echo "$1"
if [ "$1" = "" ]
then return 0
else return 1
fi
}
```

```
# test
. testare # functia se face cunoscuta
testare $1
if [ "$?" = 0 ]
then
      echo "Argumentul e corect"
else
      echo "Argumentul e incorect"
fi
```

set [**--aefhkntuvx**] [args]

Comanda permite activarea/inactivarea unor opțiuni sau pentru poziționarea parametrilor pozitionali. Starea curentă se afla in **$-**. Argumentele ramase sunt atribuite, rand pe rand, la $1, $2,.... Dacă nu exista argumente, valoarea fiecărei variabile este afișată.

Opțiuni:

-v Afișează toate liniile programului shell.

-x Afișează toate comenziile si argumentele lor pe masura ce sunt executate (prefixate de +). Folosita la depanarea programelor shell.

-e Shell-ul se oprește dacă vreo comanda returnează cod de ieșire eronat;

-n Permite testarea programelor shell, fără a le executa. De exemplu:

```
Shell
$ set unu doi trei $ set -vx
$ echo $1:$2:$3 $ set +x
unu:doi:trei
$ echo $#
3
$ echo $*
unu doi trei
```

shift [n] Deplasare spre dreapta (cu **n**) a parametrilor.

sleep n suspenda execuția pentru **n** secunde.

test conditie Comanda este echivalentă cu [conditie]. Shell-ul evaluează condiția iar dacă rezultatul evaluării este TRUE se returnează codul de ieșire 0. Condiția poate fi formată din mai multe condiții legate prin operatorii *-a* (and) și *-o* (or). Condițiile pot fi delimitate prin paranteze dar gardate de \.

times Shell-ul afișează timpul (timp utilizator si timp sistem) consumat de procesele executate de shell. De exemplu:

```
Shell
$ times
2m7s 3m7s
```

trap cmds sem Shell-ul executa comanda sau comenzile **cmds** la recepționarea semnalului **sem**. De exemplu, pentru ca la apăsarea tastei Ctr-C (semnalul 2=interrupt) sa se șteargă anumite fisiere se folosește:

```shell
$ trap 'rm $TMPFILE; exit' 2
```

Pentru a ignora un semnal se poate folosi:

```shell
$ trap "" 2
```

Pentru a reseta actiunea atasata unui semnal:

```shell
$ trap 2
```

`type` `cmds` Furnizeaza informatii despre comanda sau comenzile **cmds**. Informația specifică dacă comanda este: o funcție, un program shell, o comanda Unix standard.
De exemplu:

```shell
$ type mycd
mycd is a function
# urmeaza definitia ...
$ type pwd
pwd is a shell builtin
$ type troff
troff is /usr/bin/troff
```

`unset` `v` Pentru a șterge definirea unei variabile din mediul curent se folosește comanda **unset**. De exemplu:

```shell
$ x=100
$ echo $x
100
$ unset x
$ echo $x
# variabila x a fost stearsa
$
```

`wait` `[pid]` Așteaptă execuția procesului **pid**. Dacă **pid** este omis shell-ul așteaptă terminarea tuturor proceselor fiu. De exemplu:

```shell
$ sort fisdate > fisdate_sort &
```

```
123
$...
$ wait 123
$ # prompterul apare la terminarea procesului 123
```

Pentru a aștepta un anume proces se poate folosi variabila `$!`. De exemplu:

```Shell
prog1&
pid1=$!
...
prog2&
pid2=$!
...
wait $pid1 # se asteapta terminarea procesului pid1
...
wait $pid2 # se asteapta terminarea procesului pid2
```

4.5. Structuri de control

După execuția unei comenzi se returnează un cod de revenire, care poate fi testat în structurile de control condiționale. O comanda corect executată întoarce cod de retur zero. O comanda este o secvență de cuvinte separate prin caractere '\b' sau '\t'. Primul cuvânt specifica comanda ce se executa, iar următoarele specifica argumentele comenzii. Valoarea unei comenzi este starea ei de ieșire dacă execuția ei a decurs normal sau 200+stare (octal), dacă execuția ei sa terminat anormal.

Un lanț (pipe) de comenzi este o secvență de una sau mai multe comenzi separate prin caracterul '**|**'. Ieșirea unei comenzi servește ca intrare pentru următoarea. Fiecare comandă este un proces separat, shell-ul așteptând execuția ultimei comenzi din lant. Starea unui lanț este starea ultimei comenzi din lant.
O lista este o secvență de una sau mai multe lanțuri separate prin caracterele '**;**', '**&**' sau **&&** si **||**.
Separatorii afectează execuția:

- **;** Execuție secvențială.
- **&** Execuție asincrona (background).
- **&&** Executa următoarea lista dacă precedentul lanț a returnat starea 0.
- **||** Identic ca `&&`, dar starea de ieșire diferite de 0.

O comanda shell este o comanda simpla sau una dintre comenzile de mai jos.
a) **Structura IF** are sintaxa:

```
if lista_1
then lista
[ elif lista_2
then lista]
[ else lista]
fi
```

Se executa *lista_1*; dacă codul de retur este zero se executa lista ce urmează primului **then**. În caz contrar, se executa *lista_2* și dacă codul de retur este zero se executa lista de după al doilea **then**. De exemplu:

```shell
if test -f $1
then echo $1 este un fisier ordinar
      elif test -d $1
      then echo $1 este un director
      else echo $1 este necunoscut
fi
```

b) **Structura CASE** are sintaxa:

```
case cuvant in
sablon_1) lista;;
sablon_2) lista;;
...
esac
```

Se compara *cuvant* cu fiecare din șabloanele prezente și se executa lista de comenzi unde se constata potrivirea. De exemplu, analiza unei opțiuni din linia de comanda se poate face astfel:

```shell
case $1 in
      -r) echo optiunea r;;
      -m) echo optiunea m;;
      *) ;;
esac
```

c) **Structura FOR** are sintaxa:

```
for nume [in cuvint ...]
do lista
done
```

Variabila de bucla *nume* ia pe rand valorile din lista ce urmează lui **in**. Pentru fiecare valoare se execută ciclul **for**. Dacă **in cuvint** este omis, ciclul se executa pentru fiecare parametru pozitional actualizat. Condiția poate fi și **in *** , caz in care variabila *nume* ia pe rand ca valoare numele intrările din directorul curent. De exemplu, pentru a copia trei fișiere cu nume similare în directorul /TMP se folosește secvența:

```shell
for i in 1 2 3
do cp fis[$i] /TMP
done
```

d) **Structura WHILE** are sintaxa:

```
while lista_1
do lista
done
```

Dacă starea de ieșire din ultima comanda din *lista_1* este zero, se executa *lista*. În caz contrar, bucla se termina. De exemplu, pentru a vedea dacă o persoana este in sesiune se poate folosi secvența::

```shell
while ;
do
        if who | grep $1 /dev/null
        then echo $1 este prezent
                exit
        else
                sleep 120
        fi
done
```

Argumentul buclei **while** este comanda vidă, care returnează codul 0. Bucla se va executa pana utilizatorul, al cărui nume este dat de primul argument al procedurii, este în sesiune.

e) **Structura UNTIL** are sintaxa:

```
until lista_1
do lista
done
```

Ea este similara structurii WHILE, dar se neaga conditia de terminare a testului. De exemplu:

```shell
until who | grep $1 dev/null
do
        sleep 120
done
echo $1 este prezent
```

4.6. Redirectarea fisierelor standard

Procesul shell deschide **trei fișiere standard (cu descriptori 0,1, 2)** pentru **intrare**, **ieșire** și **ieșirea de erori**, ce sunt atribuite terminalului la care s-a deschis sesiunea respectivă. Aceste fișiere sunt asociate și utilizate într-o maniera standard de fiecare program. La execuția unei comenzi procesul asociat creat de shell mosteneste fișierele deschise, inclusiv cele standard. Acest lucru asigură independența programelor fata de dispozitivele fizice asociate de interpretor fișierelor standard.

Interpretorul permite însă redirectarea fisierelor standard de intrare/ieșire spre alte dispozitive periferice sau fișiere astfel:

< `fis` fișierul *fis* este fișierul standard de intrare;

> `fis` fișierul *fis* este fișierul standard de ieșire;

>>`fis` fișierul *fis* este fișierul standard de ieșire în adăugare;

<<`fis` fișierul standard de intrare este intrarea shell
(citește până la linia identică cu **fis**, sau pînă la sfîrșit de fișier);

<& `nr` utilizează fișierul cu descriptorul **nr** ca intrare standard;

>& `nr` similar dar pentru ieșirea standard;

0<&- sau **<&-** închide intrarea standard;

1>&- sau **>&-** închide ieșirea standard.

Exemple:

a) `$cat fis > /dev/lp` - listeaza fișierul la imprimanta;

b) `$cat f1 f2 > f3` - concateneaza fișierele *f1* și *f2* în *f3*. Dacă fișierul *f3* exista deja, prin redirectare cu > vechiul conținut este pierdut.

c) `$cat f1 f2 >> f3` - dacă *f3* exista deja, la vechiul sau conținut se adaugă rezultatul concatenarii fișierului *f1* și *f2*.

d) `$mail user1 < scris` - trimite utilizatorului *user1* o scrisoare aflată în fișierul **scris**.

e) `$echo "Invalid" >& 2` - scrie mesajul la ieșirea standard.

d) `$wc -l <<END` - executa comanda wc pana la intalnirea linie cu cuvantul 'END'.

```shell
$ wc -l <<END
>o linie
>inca una
>END
```

5. Exemple probleme rezolvate

5.1. Fișierul de comenzi de mai jos creeaza un fișier de articole, ce constituie nume de persoane:

```shell
echo Creare fisier:
echo -e “Nume fisier[fnnn]: \\c"
read fname
valid $fname
if test -f invalid
  then echo Nume invalid
  rm invalid
  exit
fi
echo > $fname
aux=0
echo Introduceti articolele:
while read string
  case $string in
   [a-zA-Z]*);;
   *) aux=1;;
  esac
  test $aux -eq 0
do echo $string >> $fname
done
sort $fname -o $fname
echo Fisierul creat:
echo
cat $fname
```

Procedura **`valid`** (următoarele linii trebuie scrise într-un fișier cu numele *valid*):

```shell
case $1 in
f[3-5][1-6][1-6][1-6]) ;;
*) echo > invalid;;
esac
```

5.2. Următorul fișier de comenzi (`pr_dir`) afișează toate fișierele dintr-un subdirector (`work`), citit ca argument din linia de comanda iar rezultatul comenzii este redirectat într-un fișier. Fișierul de comenzi se apelează astfel:

```shell
$ sh pr_dir work > /tmp/fisier_iesire
```

Conținutul fișierului **pr_dir** este următorul:

```shell
echo "cpy Fisiere de comenzi"
echo
echo $1:
echo
if test -d $1
  then
  ls -a $1/*
  for nume in $1/.[a-z,A-Z]* $1/*
  do
    if test -r $nume
      then
        echo $nume
    fi
  done
else
  cat $1
fi
```

5.3. Fișierul de comenzi de mai jos permite copierea unui fișier în alt fisier sau a mai multor fisiere într-un director.

```shell
# preia argumentele
nrargs=$#
filelist=
copylist=
# salveaza argumentele mai putin ultimul in filelist
while [ "$#" -gt 1 ]
do
  filelist="$filelist $1"
  shift
done
dest="$1"
# daca sunt: a) mai putin de doua argumente,
# sau b) mai mult de doua fara ca ultimul sa fie director
if [ "$nrargs" -lt 2 -o "$nrargs" -gt 2 -a ! -d "$dest" ]
  then
    echo "Usage: cpy file1 file2"
    echo "       cpy file(s) dir"
    exit 1;
fi
# construieste destinatia
for from in $filelist
  do
    if [ -d "$dest" ]
```

```
      then
        destfile="$dest/`basename $from`"
      else
        destfile="$dest"
    fi
# pentru fiecare fisier din filelist se verifica daca acesta
# exista si se afiseaza un mesaj de suprascriere. Daca raspunsul e
# yes sau daca fisierul nu exista acesta este adaugat la copylist.
if [ -f "$destfile" ]
  then
    echo n "$destfile already exists; Overwrite (yes/no)?:"
    read answer
    if [ "$answer" = yes ] # se lasa spatiu pentru operator!
      then
        copylist="$copylist $from"
    fi
  else
    copylist="$copylist $from"
fi
done
if [ -n "$copylist" ]
  then
    echo -e "Copy \tFrom: $copylist"
    echo -e "\tTo: $dest"
    cp $copylist $dest
fi
```

5.4. Alte exemple de comenzi:

a) exemplu de apelare al comenzii `eval` pentru evidentierea parametrilor tip line de comanda primiti de o aplicație

```shell
$ eval echo \$$#
bash
```

b) modificarea valorii unei variabile prin accesul direct la zona de memorie ocupată

```shell
$ x=100
$ px=x
$ eval echo \$$px
100
$ eval $px=5
$ echo $x
5
```

c) afișarea elementelor unui șir de caractere utilizând separatorul “:”

```shell
line="Laborator SO: Linux: Anul II"
IFS=:
set $line
echo $#
for camp; do echo $camp; done
```

d) numărarea liniilor dintr-un fișier

```shell
file=$1
count=0
while read line
do
       count=`expr $count + 1`
done < $file
echo $count
```

Nota: bucla while este executată într-un subshell deoarece intrarea ei este redirectată din variabila `$file`.

6. Bibliografie

- Advanced Bash-Scripting Guide
  http://tldp.org/LDP/abs/html/index.html
- Mihai Budiu - Elemente de programare în Shell-ul Unix
  http://www.cs.cmu.edu/~mihaib/articles/shell-2/shell-2-html.html
- UNIX shell differences and how to change your shell
  http://www.faqs.org/faqs/unix-faq/shell/shell-differences/
- BASH Frequently-Asked Questions
  http://www.faqs.org/faqs/unix-faq/shell/bash/
- http://en.wikipedia.org/wiki/Environment_variable
- William E. Shotts Jr., Writing shell scripts
  http://linuxcommand.org/lc3_writing_shell_scripts.php
- A Simple Makefile Tutorial
  https://www.cs.colby.edu/maxwell/courses/tutorials/maketutor/
- Learn Makefiles
  https://makefiletutorial.com/
- What is a Makefile and how does it work?
  https://opensource.com/article/18/8/what-how-makefile